

Tip Form

Predstavlja kontrolu forme, koja je prozor koji može da prikazuje druge kontrole poput labela, tekstualnih polja i dugmadi sa kojima korisnik interaguje.

Lakše je koristiti dizajner forme za dodavanje novog obrasca i njegovo čuvanje u XAML datoteci.

Takođe je moguće kreirati novu formu tokom izvršavanja, pozivanjem metode `Forms.AddForm`. Takođe je moguće pozvati metodu `Forms.LoadForm` da biste učitali formu iz XAML datoteke koji sadrži njegov dizajn.

Formu možete prikazati pozivanjem metode `Form.Show`.

Kontrola forme nasleđuje sva svojstva, metode i događaje tipa `Control`.

Pored toga, kontrola Forma ima neke nove članove koji su navedeni u nastavku:

Me:

Možete koristiti „Me“ da biste se pozvali na trenutni Formu, tako da ako imate posla sa formom pod nazivom `Forma1`, možete koristiti obe sledeće izjave da biste promenili njen naslov:

```
Form1.Text = "Moje ime je Forma1"  
Ja.Text = "Ja sam takođe Forma1"
```

`Form.AddButton(buttonName, left, top, width, height) As Button`

Dodaje novu kontrolu Buton – *Dugme* formi tokom izvršavanja.

Lakše je koristiti dizajner obrazaca za kreiranje kontrola i podešavanje njihovih lokacija i veličina tokom dizajniranja, zato nemojte koristiti ovu metodu osim ako vam je zaista potrebno.

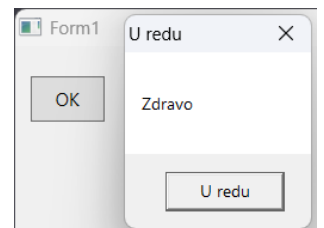
Parametri:

- *buttonName*: Jedinstveno ime za novo Dugme.
- *left*: X-pozicija kontrole.
- *top*: Y-pozicija kontrole.
- *width*: Širina kontrole. Koristite -1 za automatsku širinu.
- *height*: Visina kontrole. Koristite -1 za automatsku visinu.

Vraća: Ključ dugmeta u formuli `{formName.buttonName}` kao što je `"form1.button1"`. sVB može da obrađuje ovaj ključ kao objekat tipa `Button`, tako da preko njega možete pristupiti svojstvima, metodama i događajima dugmeta.

Primer: Dodajte ovaj kôd u globalno područje forme.

```
BtnOK = Me.AddButton("Button1", 10, 20, 50, 30)  
BtnOK.Text = "OK"  
BtnOK.OnClick = BtnOK_OnClick  
Sub BtnOK_OnClick()  
Me.ShowMessage("Zdravo", "U redu")  
EndSub
```



Pritisnite F5 da biste pokrenuli program. Dugme OK će biti prikazano na formi, a kada kliknete na njega, prikazaće se okvir sa porukom "Zdravo":

`Form.AddCheckBox(checkBoxName, left, top, text, checked) As CheckBox`

Dodaje novu kontrolu `CheckBox` na formu.

Lakše je koristiti dizajner obrazaca za kreiranje kontrola i podešavanje njihovih lokacija i

veličina tokom dizajniranja, zato nemojte koristiti ovu metodu osim ako vam je zaista potrebno.

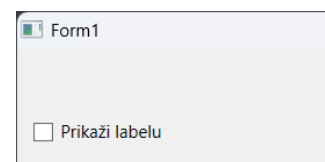
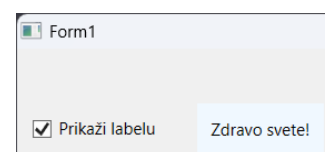
Parametri:

- *checkBoxName*: Jedinstveno ime za novi CheckBox.
- *left*: X-pozicija kontrole.
- *top*: Y-pozicija kontrole.
- *text*: Tekst koji se prikazuje na CheckBox-u.
- *checked*: Vrednost koju treba postaviti na svojstvo CheckBox.Checked.

Vraća: Ključ CheckBox-a u formuli {formName.checkBoxName} kao što je "form1.checkBox1". sVB može da obrađuje ovaj ključ kao objekat tipa CheckBox, tako da možete pristupiti svojstvima, metodama i događajima CheckBox-a preko njega.

Primer: Dodajte ovaj kôd u globalno područje forme.

```
' Dodaj polje za potvrdu i obradite njegov događaj OnCheck:
CheckBox1 = Me.AddCheckBox("CheckBox1", 10, 50,
"Prikaži labelu", True)
CheckBox1.OnCheck = CheckBox1_OnCheck
' Dodaj labelu levo od polja za potvrdu:
Label1 = Me.AddLabel(
"Label1",
CheckBox1.Left + CheckBox1.Width + 30,
CheckBox1.Top, -1, -1)
' Podesi svojstva labele:
Label1.BackColor = Colors.AliceBlue
Label1.Text = "Zdravo svete!"
' Poravnaj sredine labele i polja za potvrdu horizontalno:
Label1.Top = Label1.Top -
(Label1.Height - CheckBox1.Height) / 2
' Funkcija OnCheck sakriva ili prikazuje labelu:
Sub CheckBox1_OnCheck()
Label1.Visible = CheckBox1.Checked
EndSub
```



Pritisnite F5 da biste pokrenuli program. Forma će prikazati polje za potvrdu i labelu. Označite i opozovite izbor polja za potvrdu da biste prikazali ili sakrili labelu.

Form.AddComboBox(comboBoxName, left, top, width, height) As ListBox

Dodaje novu kontrolu ComboBox u formu.

Lakše je koristiti dizajner formulara za kreiranje kontrola i podešavanje njihovih lokacija i veličina tokom dizajniranja, pa nemojte koristiti ovu metodu osim ako vam je zaista potrebno.

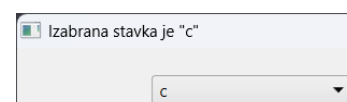
Parametri:

- *comboBoxName*: Jedinstveno ime za novo kombinovano polje.
- *left*: X-pozicija kontrole.
- *top*: Y-pozicija kontrole.
- *width*: Širina kontrole. Koristite -1 za automatsku širinu.
- *height*: Visina kontrole. Koristite -1 za automatsku visinu.

Vraća: Ključ kombinovanog okvira u formuli {formName.comboBoxName} kao što je "form1.combobox1". sVB može da obrađuje ovaj ključ kao objekat tipa ComboBox, tako da možete koristiti svojstva, metode i događaje ComboBox-a preko njega.

Primer: Dodajte ovaj kôd u globalno područje forme.

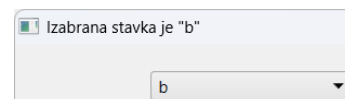
```
ComboBox1 = Me.AddComboBox(
"ComboBox1", 100, 20, 150, -1)
ComboBox1.AddItem({"a", "b", "c"})
ComboBox1.OnSelection = ComboBox1_Select
ComboBox1.SelectedIndex = 3
```



```

Sub ComboBox1_Select()
Me.Text = Text.Format(
"Izabrana stavka je [1][2][1]",
{Chars.Quote, ComboBox1.SelectedItem}
)
EndSub

```



Pritisnite F5 da biste pokrenuli program. Forma će prikazati kombinovani okvir sa poslednjom izabranom stavkom, a naslovna traka forme će prikazivati izabranu stavku i biće ažurirana svaki put kada promenite izabranu stavku.

Form.AddDatePicker(datePickerName, left, top, width, selectedDate) As DatePicker

Dodaje novu kontrolu DatePicker u formu.

Lakše je koristiti dizajner obrazaca za kreiranje kontrola i podešavanje njihovih lokacija i veličina tokom dizajniranja, zato nemojte koristiti ovu metodu osim ako vam je zaista potrebno.

Parametri:

- *datePickerName*: Jedinstveno ime za novu kontrolu DatePicker.
- *left*: X-pozicija kontrole.
- *top*: Y-pozicija kontrole.
- *width*: Širina kontrole. Koristite -1 za automatsku širinu.
- *selectedDate*: Datum koji će biti izabran u kontroli kada se prikaže.

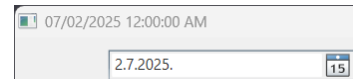
Vraća: Ključ kontrole DatePicker u formuli {formName.datePikkerName} kao što je "form1.datepicker1". sVB može da obrađuje ovaj ključ kao objekat tipa DatePicker, tako da možete pristupiti svojstvima, metodama i događajima kontrole DatePicker preko njega.

Primer: Dodajte ovaj kôd u globalno područje forme.

```

DatePicker1 = Me.AddDatePicker(
"DatePicker1", 75, 10, 200, #12/18/2022#)
DatePicker1.OnSelection = DatePicker1_OnSelection
Sub DatePicker1_OnSelection()
Me.Text = DatePicker1.SelectedDate
EndSub

```



Pritisnite F5 da biste pokrenuli program. Datum 18.12.2022. će biti izabran u biraču datuma i svaki put kada promenite izabrani datum, on će biti prikazan u naslovnoj traci forme.

Form.AddLabel(labelName, left, top, width, height) As Label

Dodaje novu kontrolu Label formi.

Lakše je koristiti dizajner forme za kreiranje kontrola i podešavanje njihovih lokacija i veličina tokom dizajniranja, pa nemojte koristiti ovu metodu osim ako vam zaista nije potrebno.

Parametri:

- *labelName*: Jedinstveno ime za novu oznaku (Label).
- *left*: X-pozicija kontrole.
- *top*: Y-pozicija kontrole.
- *width*: Širina kontrole. Koristite -1 za automatsku širinu.
- *height*: Visina kontrole. Koristite -1 za automatsku visinu.

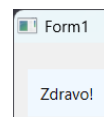
Vraća: Ključ oznake (label) u formuli {formName.labelName} kao „form1.label1“. sVB može da obrađuje ovaj ključ kao objekat tipa Oznaka (Label), tako da možete pristupiti svojstvima, metodama i događajima oznake (Label) preko njega.

Primer: Dodajte ovaj kôd u globalno područje forme.

```

Label1 = Me.AddLabel("Label1", 10, 20, -1, -1)
Label1.Text = "Zdravo!"
Label1.OnMouseEnter = Label1_OnMouseEnter
Label1.OnMouseLeave = Label1_OnMouseleave
Sub Label1_OnMouseEnter()
Label1.BackColor = Colors.AliceBlue
EndSub
Sub Label1_OnMouseleave()
Label1.BackColor = Colors.Transparent
EndSub

```



Pritisnite F5 da biste pokrenuli program. Forma će prikazati oznaku „Zdravo“, a kada pokazuje miša uđe u njenu oblast, njena pozadinska boja će se promeniti, gde možete potvrditi da njena širina i visina odgovaraju njenom sadržaju (jer smo koristili -1 za obe).

Kada pokazivač miša napusti oblast oznake, njena pozadinska boja će ponovo biti providna. To možete ponoviti koliko god puta želite!

Form.AddListBox(listBoxName, left, top, width, height) As ListBox

Dodaje novu kontrolu ListBox formi.

Lakše je koristiti dizajner obrazaca za kreiranje kontrola i podešavanje njihovih lokacija i veličina tokom dizajniranja, zato nemojte koristiti ovu metodu osim ako vam je zaista potrebno.

Parametri:

- *listBoxName*: Jedinstveno ime za novi ListBox.
- *left*: X-pozicija kontrole.
- *top*: Y-pozicija kontrole.
- *width*: Širina kontrole. Koristite -1 za automatsku širinu.
- *height*: Visina kontrole. Koristite -1 za automatsku visinu.

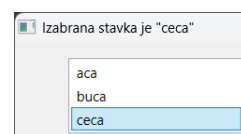
Vraća: Ključ liste u formuli {formName.listBoxName} kao što je "form1.listbox1". sVB može da obrađuje ovaj ključ kao objekat tipa ListBox, tako da možete pristupiti svojstvima, metodama i događajima ListBox-a preko njega.

Primer: Dodajte ovaj kôd u globalno područje forme.

```

ListBox1 = Me.AddListBox(
"ListBox1", 100, 20, 150, -1)
ListBox1.AddItem({"aca", "buca", "ceca"})
ListBox1.OnSelection = ListBox1_Select
ListBox1.SelectedIndex = 3
Sub ListBox1_Select()
Me.Text = Text.Format(
"Izabrana stavka je [1][2][1]",
{Chars.Quote, ListBox1.SelectedItem}
)
EndSub

```



Pritisnite F5 da biste pokrenuli program. Forma će prikazati listu sa poslednjom izabranom stavkom, a naslovna traka forme će prikazivati izabranu stavku i biće ažurirana svaki put kada promenite izabranu stavku.

Form.AddMainMenu(menuName) As MainMenu

Dodaje kontrolu MainMenu trenutnom obrascu. Ako već postoji glavni meni, biće zamenjen.

Parametar:

- *menuName*: Naziv glavnog menija.

Vraća: Ključ menija u formuli {formName.mainMenuName} kao što je "form1.mainMenu1".

sVB može da obrađuje ovaj ključ kao objekat tipa MainMenu, tako da preko njega možete pristupiti svojstvima, metodama i događajima MainMenu-a.

Primer: Možete videti ovu metodu u akciji u projektu "sVB Notepad" u folderu sa primerima. Koristi se u obrascu FrmMain za kreiranje glavnog menija pomoću koda, jer dizajner forme ne podržava vizuelno dodavanje menija:

```
M = Me.AddMainMenu("mainmenu")
```

Zatim se promenljiva glavnog menija M koristi za dodavanje stavki menija kao što možete videti u ostatku koda.

Form.AddProgressBar(progressBarName, left, top, width, height, minimum, maximum) As ProgressBar

Dodaje novu ProgressBar kontrolu u formu.

Lakše je koristiti dizajner formi za kreiranje kontrola i podešavanje njihovih lokacija i veličina tokom dizajniranja, zato nemojte koristiti ovu metodu osim ako vam zaista nije potrebno.

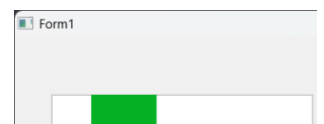
Parametri:

- *progressBarName*: Jedinstveno ime za novu ProgressBar.
- *left*: X-pozicija kontrole.
- *top*: Y-pozicija kontrole.
- *width*: Širina kontrole. Koristite -1 za automatsku širinu.
- *height*: Visina kontrole. Koristite -1 za automatsku visinu.
- *minimum*: Minimum vrednosti napretka.
- *maximum*: Maximum vrednosti napretka. Koristite 0 ako je maksimalna vrednost neodređena. Ovo će prikazati beskonačno pokretni okvir za odabir.

Vraća: Ključ ProgressBar-a u formuli {formName.progressBar} kao "form1.progressBar1". sVB može da obrađuje ovaj ključ kao objekat tipa ProgressBar, tako da preko njega možete pristupiti svojstvima, metodama i događajima ProgressBar-a.

Primer: Dodajte ovaj kôd u globalno područje forme.

```
ProgressBar1 = Me.AddProgressBar(  
"ProgressBar1",  
35, 55, 250, 30,  
0, 0  
)
```



Pritisnite F5 da biste pokrenuli program. Forma će prikazati traku napretka sa beskrajno pokretnim okvirom.

Form.AddRadioButton(radioButtonName, left, top, text, groupName, checked) As RadioButton

Dodaje novu kontrolu RadioButton formi.

Lakše je koristiti dizajner obrazaca za kreiranje kontrola i podešavanje njihovih lokacija i veličina tokom dizajniranja, zato nemojte koristiti ovu metodu osim ako vam je zaista potrebno.

Parametri:

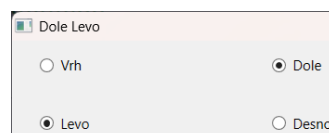
- *radioButtonName*: Jedinstveno ime za novo RadioButton.
- *left*: X-pozicija kontrole.
- *top*: Y-pozicija kontrole.

- *text*: Tekst koji se prikazuje na RadioButton-u.
- *groupName*: Ime grupe kojoj se dodaje dugme.
- *checked*: Vrednost koju treba postaviti na svojstvo Checked.

Vraća: Ključ RadioButton-a u formuli {formName.radioButtonName} kao što je "form1.radioButton1". sVB može da obrađuje ovaj ključ kao objekat tipa RadioButton, tako da možete pristupiti svojstvima, metodama i događajima RadioButton-a preko njega.

Primer: Dodajte ovaj kôd u globalno područje forme. Kreira 2 grupe radio dugmadi, a svaka sadrži dva dugmeta:

```
' Radio dugmad grupa1
RdoTop = Me.AddRadioButton(
"RdoTop", 35, 30, "Vrh", "grupa1", False)
RdoBottom = Me.AddRadioButton(
"RdoBottom", 235, 30, "Dole", "grupa1", True)
' Radio dugmad grupa2
RdoLeft = Me.AddRadioButton(
"RdoLeft", 35, 80, "Levo", "grupa2", True)
RdoRight = Me.AddRadioButton(
"RdoRight", 235, 80, "Desno", "grupa2", False)
' Samo jedan obrađivač događaja za sva radio dugmad:
RdoTop.OnCheck = OnCheck
RdoBottom.OnCheck = OnCheck
RdoLeft.OnCheck = OnCheck
RdoRight.OnCheck = OnCheck
' Pozovi obrađivač jednom da biste obrađivali početna radio dugmad
' stanja koja su podešena metodom AddRadioButton
OnCheck()
Sub OnCheck()
pos = ""
If RdoTop.Checked Then
pos = "Vrh"
Else
pos = "Dole"
EndIf
If RdoLeft.Checked Then
pos = pos + " Levo"
Else
pos = pos + " Desno"
EndIf
Me.Text = pos
EndSub
```



Pritisnite F5 da biste pokrenuli program. Forma će prikazati 4 radio dugmad sa izabranim radio dugmadima „Dole“ i „Levo“.

Naslovna traka forme će prikazivati izabranu poziciju i ažuriraće se svaki put kada promenite izabrana radio dugmad.

Form.AddScrollBar(scrollBarName, left, top, width, height, minimum, maximum, value) As ScrollBar

Dodaje novu kontrolu ScrollBar formi.

Lakše je koristiti dizajner obrazaca za kreiranje kontrola i podešavanje njihovih lokacija i veličina tokom dizajniranja, zato nemojte koristiti ovu metodu osim ako vam je zaista potrebno.

Parametri:

- *scrollBarName*: Jedinstveno ime za novu traku za skrolovanje.
- *left*: X-pozicija kontrole.
- *top*: Y-pozicija kontrole.
- *width*: Širina kontrole. Koristite -1 za automatsku širinu.

- *height*: Visina kontrole. Koristite -1 za automatsku visinu.
- *minimum*: Minimalna vrednost trake za skrolovanje
- *maximum*: Maksimalna vrednost trake za skrolovanje.
- *value*: Trenutna vrednost trake za skrolovanje.

Vraća: Ključ trake za skrolovanje u formuli {formName.scrollbarName} kao što je "form1.scrollbar1". sVB može da obrađuje ovaj ključ kao objekat tipa Scrollbar, tako da možete pristupiti svojstvima, metodama i događajima trake za skrolovanje preko njega.

Primer: Dodajte ovaj kôd u globalno područje forme:

```
ScrollBar1 = Me.AddScrollBar(
"ScrollBar1",
260, -175, -1, 500,
0, 200, 50
)
' Rotiraj traku za skrolovanje da bude horizontalna
ScrollBar1.Angle = -90
ScrollBar1.BackColor = Colors.White
ScrollBar1.OnScroll = ScrollBar1_OnScroll
Me.Text = 50
Sub ScrollBar1_OnScroll()
Me.Text = Math.Round(ScrollBar1.Value)
EndSub
```



Pritisnite F5 da biste pokrenuli program. Forma će prikazati traku za skrolovanje, a naslovna traka forme će pokazati njenu vrednost (što je naznačeno položajem palca). Naslov forme će se ažurirati svaki put kada promenite položaj palca na traci za skrolovanje.

Form.AddSlider(sliderName, left, top, width, height, minimum, maximum, value, tickFrequency) As Slider

Dodaje novu kontrolu Klizača u formu.

Lakše je koristiti dizajner obrazaca za kreiranje kontrola i podešavanje njihovih lokacija i veličina tokom dizajniranja, zato nemojte koristiti ovu metodu osim ako vam je zaista potrebno.

Parametri:

- *sliderName*: Jedinstveno ime za novi Klizač.
- *left*: X-pozicija kontrole.
- *top*: Y-pozicija kontrole.
- *width*: Širina kontrole. Koristite -1 za automatsku širinu.
- *height*: Visina kontrole. Koristite -1 za automatsku visinu.
- *minimum*: Minimalna vrednost klizača
- *maximum*: Maksimalna vrednost klizača.
- *value*: Trenutna vrednost klizača.
- *tickFrequency*: Rastojanje između oznaka na slajdu.

Vraća: Ključ klizača u formuli {formName.sliderName} kao što je "form1.slider1". sVB može da obrađuje ovaj ključ kao objekat tipa Klizač, tako da preko njega možete pristupiti svojstvima, metodama i događajima klizača.

Primer: Dodajte ovaj kôd u globalno područje forme:

```
Slider1 = Me.AddSlider(
"Slider1",
20, 50, 500, 30,
0, 100, 50, 10
)
Slider1.ForeColor = Colors.Red
```



```

Slider1.OnSlide = Slider1_OnSlide
Me.Text = 50
Sub Slider1_OnSlide()
Me.Text = Math.Round(Slider1.Value)
EndSub

```

Pritisnite F5 da biste pokrenuli program. Forma će prikazati klizač, a naslovna traka forme će prikazati njegovu vrednost (što je naznačeno položajem palca). Naslov obrasca će se ažurirati svaki put kada pomerite palac na klizaču.

Form.AddTextBox(textBoxName, left, top, width, height) As TextBox

Dodaje novu kontrolu TextBox formi.

Lakše je koristiti dizajner obrazaca za kreiranje kontrola i podešavanje njihovih lokacija i veličina tokom dizajniranja, zato nemojte koristiti ovu metodu osim ako vam je zaista potrebno.

Parametri:

- *textBoxName*: Jedinstveno ime za novu TextBox.
- *left*: X-pozicija kontrole.
- *top*: Y-pozicija kontrole.
- *width*: Širina kontrole. Koristite -1 za automatsku širinu.
- *height*: Visina kontrole. Koristite -1 za automatsku visinu. - 540 -

Vraća: Ključ TextBox-a u formuli {formName.textBoxName} kao što je "form1.textbox1". sVB može da obrađuje sa ovim ključem kao objektom tipa TextBox, tako da možete pristupiti svojstvima, metodama i događajima TextBox-a preko njega.

Primer: Dodajte ovaj kôd u globalno područje forme:

```

TextBox1 = Me.AddTextBox(
"TextBox1", 10, 15, -1, -1)
TextBox1.Text = "Zdravo"

```



Pritisnite F5 da biste pokrenuli program. Forma će prikazati tekstualno polje kao što je prikazano na slici:

Form.AddTimer(timerName, interval) As WinTimer

Dodaje novu WinTimer kontrolu formi tokom izvršavanja.

Ne možete koristiti dizajner forme za dodavanje tajmera formi tokom dizajniranja.

Parametri:

- *timerName*: Jedinstveno ime za novi tajmer.
- *interval*: Vreme kašnjenja u milisekundama između otkucaja.

Vraća: Ključ tajmera u formuli {formName.timerName} kao što je "form1.timer1". sVB može da obrađuje ovaj ključ kao objekat tipa WinTimer, tako da možete pristupiti svojstvima, metodama i događajima WinTimer-a preko njega.

Primer: Možete videti ovu metodu u akciji u projektu „Stop Watch“ – *Štoperica* u folderu sa primerima. Koristi se za dodavanje dva tajmera u formu, jedan za prikazivanje sata u naslovnoj traci, a drugi za ažuriranje prikaza štoperice:

```

ClockTimer = Me.AddTimer("clockTimer", 1000)
ClockTimer.OnTick = ShowClock
StopWatch = Me.AddTimer("StopWatch", 10)
StopWatch.Pause()
StopWatch.OnTick = ShowStopWatch

```


Form.AddToggleButton(toggleButtonName, left, top, width, height) As ToggleButton

Dodaje novu kontrolu ToggleButton u formu.

Lakše je koristiti dizajner obrazaca za kreiranje kontrola i podešavanje njihovih lokacija i veličina tokom dizajniranja, pa nemojte koristiti ovu metodu osim ako vam je zaista potrebno.

Parametri:

- *toggleButtonName*: Jedinstveno ime za novo ToggleButton.
- *left*: X-pozicija kontrole.
- *top*: Y-pozicija kontrole.
- *width*: Širina kontrole. Koristite -1 za automatsku širinu.
- *height*: Visina kontrole. Koristite -1 za automatsku visinu.

Vraća: Ključ dugmeta za prebacivanje (ToggleButton) u formuli {formName.toggleButtonName} kao „form1.toggleButton1“. sVB može da obrađuje ovaj ključ kao objekat tipa ToggleButton, tako da možete pristupiti svojstvima, metodama i događajima dugmeta za prebacivanje (ToggleButton) preko njega.

Primer: Dodajte ovaj kôd u globalno područje forme:

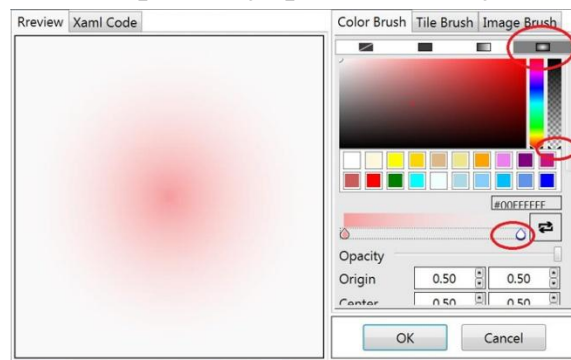
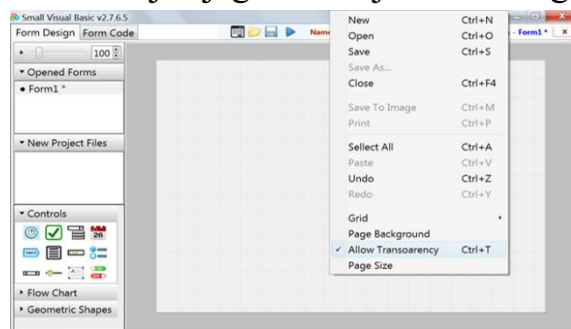
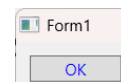
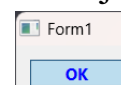
```
ToggleButton1 = Me.AddToggleButton(  
"ToggleButton1", 10, 10, 70, 20)  
ToggleButton1.Text = "OK"  
ToggleButton1.ForeColor = Colors.Blue  
ToggleButton1.OnCheck = ToggleButton1_OnCheck  
ToggleButton1.Checked = True  
Sub ToggleButton1_OnCheck()  
ToggleButton1.FontBold = ToggleButton1.Checked  
EndSub
```

Pritisnite F5 da biste pokrenuli program. Forma će prikazati dugme za prebacivanje i biće označeno. Kliknite na njega da biste promenili njegovo stanje označenog i primetite da se njegova debljina fonta menja iz podebljanog u normalno i obrnuto kako se menja njegovo stanje označenog.

Form.AllowTransparency()

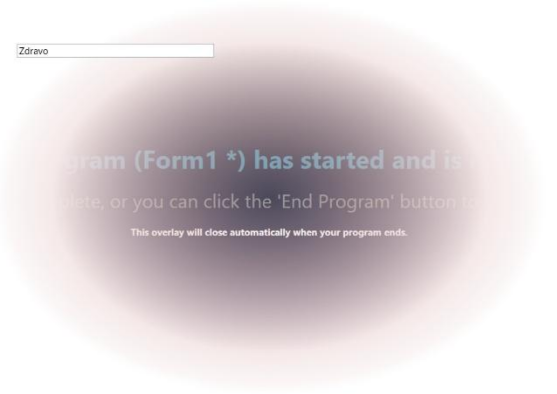
Pozovite ovu metodu da biste omogućili korišćenje transparentnih boja na pozadini forme. Ovo morate da uradite pre nego što se forma prikaže, što važi samo kada kreirate formu pozivanjem metoda Forms.AddForm ili Forms.LoadForm, i pre nego što pozovete metod Form.Show. Forme koje je kreirao dizajner obrazaca neće biti pogođeni pozivanjem metode AllowTransparency, jer se obrazac prikazuje pre izvršavanja bilo kog koda koji napišete u globalnom delu obrasca. U tom slučaju, morate koristiti dizajner obrazaca da biste omogućili transparentnost obrasca, tako što ćete kliknuti desnim tasterom miša na površinu obrasca i označiti stavku kontekstnog menija „Allow transparency“ – *Dozvoli transparentnost*. Ovo će dodati poziv metode Form.AllowTransparency u kodu generisanom iza u datoteci form.sb.gen, pre pozivanja metode Form.Show.

Primer: Možete videti ovu metodu u akciji u projektu FormShape u folderu sa primerima. Koristili smo „Dozvoli transparentnost“ u dizajneru obrazaca



da bismo omogućili transparentnost obrasca, a zatim smo koristili stavku kontekstnog menija „Page Background“ – *Pozadina stranice* da bismo promenili pozadinu forme u radialnu gradijentnu četkicu, dajući njenom spoljnom kraju transparentnu boju kako bismo obrisali pozadinu obrasca oko elipse:

Pritisnite F5 da biste pokrenuli ovaj projekat. Imaćete eliptični oblik sa gradijentnom transparentnošću i možete ga prevući iz bilo koje tačke elipse da biste ga pomerili po ekranu, ali tačke van elipse ne postoje, a klik van elipse će preneti klik na bilo koji objekat iza obrasca, kao što je radna površina prozora ili drugi prozor!



Form.ArgsArr As Array

Vraća dodatne podatke poslate formi putem drugog argumenta metoda Forms.ShowForm, Forms.ShowDialog i Form.ShowChildForm.

Uprkos svom imenu, ovom svojstvu možete poslati string, broj ili kontrolu, ne samo niz. Uvek imajte na umu da je sVB dinamički jezik i da svaka promenljiva može da sadrži bilo koji tip podataka.

Primeri: Ovu metodu možete videti u akciji u projektu „Show Dialogs“ u folderu sa primerima. Koristi se u formi FrmMsg.sb da bi se dobio naslov i tekst poruke koji će se prikazivati u okviru za poruku:

```
Info = Me.ArgsArr  
Me.Text = Info!Title  
LblMsg.Text = Info!Message
```

Da bismo prosledili ove dve informacije, poslali smo dinamički objekat drugom argumentu metode ShowDialog. Ovaj kôd možete pronaći u funkciji MsgBox u datoteci Global.sb:

```
info!Title = strTitle  
info!Message = strMessage  
Return Forms.ShowDialog("FrmMsg", info)
```

ArgsArr se takođe koristi u formi FrmInput.sb da bi se dobio tekst poruke koji će se prikazati u polju za unos:

```
LblMsg.Text = Me.ArgsArr
```

Ovo je samo jedna informacija, tako da se prosleđuje kao jedan string drugom argumentu metode ShowDialog. Ovaj kod možete pronaći u funkciji InputBox u datoteci Global.sb:

```
result = Forms.ShowDialog("FrmInput", message)
```

Form.Close()

Zatvara formu. Ovo će obrisati sve podatke prikazane njegovim kontrolama, jer će forma zapravo biti potpuno uklonjena. Ne možete pozvati metodu Form.Show da biste prikazali formu nakon što je zatvorena, osim ako prethodno niste ponovo kreirali novu instancu. Ako želite da forma ostane aktivna, ali samo da je sakrijete, možete jednostavno postaviti njegovo svojstvo Form.Visible na False ili pozvati metod Form.Hide da biste ga sakrili, tako da ga i dalje možete ponovo prikazati.

Imajte na umu da će zatvaranje glavne forme projekta zatvoriti celu aplikaciju. Možete izabrati

glavnu formu tako što ćete jednostavno otvoriti bilo koju formu projekta u sVB-u i pritisnuti F5 da biste pokrenuli projekat, tako da će ta forma biti prvi prikazana kada se aplikacija pokrene.

Form.ContainsControl(controlName) As Boolean

Vraća True ako obrazac prikazuje kontrolu sa datim imenom.

Parametar:

- *controlName*: Ime kontrole koju treba tražiti. Nije osetljivo na velika i mala slova.

Vraća: True ili False

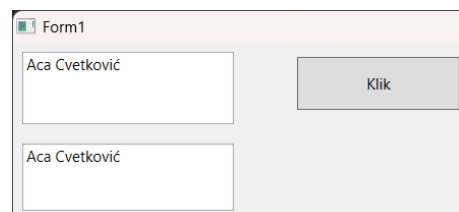
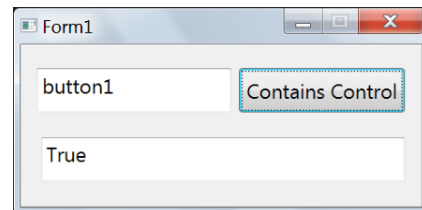
Primer: Možete testirati ovu metodu tako što ćete dizajnirati formu kao na slici:

Zatim dodajte sledeći kôd u funkciju za obradu događaja Button1_Click:

```
TextBox2.Text = Me.ContainsControl(TextBox1.Text)
```

Pritisnite F5 da biste pokrenuli projekat i testirali neka imena, kao što su prazan string, textbox1, button1 i btnTest. **Čemu ovo kad ne radi!?** Kad sve to može prosto:

```
TextBox2.Text = TextBox1.Text
```



Form.Controls As Array

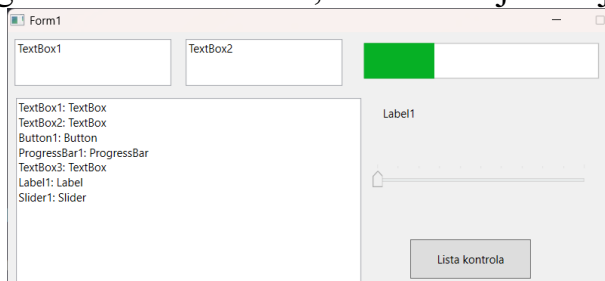
Vraća niz koji sadrži imena svih kontrola prikazanih na formu.

Poželjno je koristiti ovo svojstvo u petlji ForEach, sa odgovarajućim imenom promenljive iteratora petlje koje počinje ili se završava rečju Control, tako da ga možete koristiti kao objekat kontrole.

Primer: Dodajte tekstualno polje, dugme i neke druge kontrole na formu, zatim dodajte ovaj kôd u obrađivač događaja klika na dugme, koji će prikazati sva imena i tipove kontrola u tekstualnom polju:

```
ForEach control1 In Me.Controls  
    TextBox1.Append(control1.Name)  
    TextBox1.Append(": ")  
    TextBox1.AppendLine(control1.TypeName)  
Next
```

Pritisnite F5 da biste pokrenuli projekat i kliknite na dugme. Tekstualno polje će prikazati informacije poput ovih:



Form.DialogResult As DialogResult

Postavlja ime dugmeta na koje je korisnik kliknuo kada je zatvorio dijalošku formu.

Podrazumevana vrednost ovog svojstva je DialogResult.Cancel, što je zapravo string „Cancel“.

Imajte na umu da ne možete direktno pročitati ovo svojstvo nakon zatvaranja dijaloške forme, već njegovu vrednost vraća metod Forms.ShowDialog. Pogledajte primer tipa DialogResult.

Primer: Tip DialogResult sadrži najpoznatija imena dugmadi, ali možete koristiti i bilo koje prilagođeno ime koje želite, ili čak koristiti niz ili dinamički objekat koji nosi bilo koje vrednosti koje želite kao rezultat iz dijaloške forme. Ovo možete videti u datoteci InputBox.sb u projektu „Dialogs“ u folderu sa primerima, gde želimo da prosledimo ulazni tekst i ime dugmeta zajedno kao rezultat dijaloga:

```

Sub BtnOK_OnClick()
result!InputText = TxtInput.Text
result!Button = DialogResult.OK
Me.DialogResult = result
Me.Close()
EndSub

```

Ovaj rezultat dobijamo u funkciji InputBox u datoteci Global.sb ovako:

```

result = Forms.ShowDialog("FrmInput", message)
If result = DialogResult.Cancel Then
Return ""
Else
Return result!InputText
EndIf

```

Form.Hide()

Skriva formu. Zapravo postavlja svojstvo Form.Visible na False.

Formu možete ponovo prikazati pozivanjem metode Form.Show.

Form.Icon

Dobija ili postavlja putanju datoteke slike koja će se prikazati kao ikona u naslovnoj traci forme.

Form.IsLoaded As Boolean

Vraća True ako je trenutna forma učitana, bez obzira da li je prikazana u tom trenutku ili ne.

Form.OnClosed

Ovaj događaj se pokreće nakon što se forma zatvori. Ovaj događaj možete koristiti za odlaganje bilo kojih resursa ili čuvanje bilo kojih podešavanja.

Za više informacija, pročitajte o rukovanju kontrolnim događajima.

Form.OnClosing

Ovaj događaj se pokreće neposredno pre nego što se forma zatvori.

Možete postaviti svojstvo Event.Handled na True da biste otkazali zatvaranje forme.

Za više informacija, pročitajte o rukovanju kontrolnim događajima.

Primer: Možete videti ovaj događaj u akciji u projektu „sVB Notepad“ u folderu sa primerima. Koristi se u formi frmMain.sb za pozivanje AskToSave da bi se korisnik zamolio da sačuva izmene pre zatvaranja forme. Ako korisnik izabere dugme za otkazivanje, funkcija AskToSave vraća vrednost False, što postavlja svojstvo Event.Handled na True, pa otkazuje zatvaranje forme.

```

Sub FrmMain_OnClosing()
Event.Handled = (AskToSave() = False)
EndSub
Function AskToSave()
If IsModified Then
result = Dialogs.MsgBox(
"Sačuvaj izmene",
"Da li želite da sačuvate izmene?"
)
If result = DialogResult.Yes Then
If Save() = False Then
Return False

```

```

EndIf
ElseIf result = DialogResult.Cancel Then
Return False
EndIf
EndIf
Return True
EndFunction

```

Form.OnPreviewKeyDown

Ovaj događaj se pokreće kada korisnik pritisne taster na formi ili bilo kojoj od njenih podređenih kontrola. Tipična upotreba je rukovanje ovim događajem kako bi se odgovorilo na sve pritisnute tastere čak i kada je kontrola na formi u fokusu, kao kada korisnik kuca u tekstualno polje. Ovo vam omogućava da rukujete prečicama na tastaturi za menije i druge funkcionalnosti. Koristite svojstva tastature da biste dobili informacije o tasteru koji je pokrenuo događaj i stanju tastera Ctrl, Shift, Alt i drugih specijalnih tastera. Imajte na umu da pritiskanje F10 ima isti efekat kao i pritiskanje Alt, pa ako želite da koristite F10 u drugom zadatku u vašem programu, ne zaboravite da podesite svojstvo `Event.Handled` na `True` da biste sprečili Windows da dalje obrađuje taster F10 nakon što izvršite zadatak. Za više informacija, pročitajte o rukovanju događajima kontrole.

Primer: Možete videti ovo svojstvo u akciji u projektu 'sVB Notepad' u folderu sa primerima. Koristi se u glavnoj formi (pod nazivom `FrmMain`) za izvršavanje prečica na tastaturi za menije. Ove prečice uključuju pritiskanje kontrolnog tastera kao što je `Ctrl+N` za otvaranje novog dokumenta, tako da prvo proveravamo da li je kontrolni taster pritisnut, u suprotnom izlazimo iz potprograma. Zatim možemo proveriti poslednji pritisnuti taster da bismo izvršili potrebnu radnju. Ovo je deo ovog obrađivača događaja, a ceo kôd možete videti u projektu:

```

Sub FrmMain_OnPreviewKeyDown()
If Keyboard.CtrlPressed = False Then
Return
EndIf
key = Event.LastKey
If key = Keys.N Then
MenuNew_OnClick()
Event.Handled = True
ElseIf key = Keys.O Then
MenuOpen_OnClick()
Event.Handled = True
ElseIf key = Keys.S Then
MenuSave_OnClick()
Event.Handled = True
EndIf
EndSub

```

Form.OnPreviewKeyUp

Ovaj događaj se pokreće kada korisnik otpusti taster na formi ili bilo kojoj od njenih podređenih kontrola. Tipična upotreba je da se ovaj događaj obrađuje tako da reaguje na sve pritisnute tastere čak i kada je kontrola na formi u fokusu, kao kada korisnik kuca u tekstualno polje. Ovo vam omogućava da obrađujete prečice na tastaturi za menije i druge funkcionalnosti (pogledajte primer događaja `OnPreviewKeyDown`).

Koristite svojstva tastature da biste dobili informacije o tasteru koji je pokrenuo događaj i stanju tastera Ctrl, Shift, Alt i drugih specijalnih tastera. Za više informacija, pročitajte o rukovanju događajima kontrole.

Form.OnPreviewMouseWheel

Ovaj događaj se više puta pokreće dok korisnik pomera točak miša preko forme ili bilo koje njene kontrole. Forma takođe može da obrađuje događaj `Control.OnMouseWheel`, ali se neće pokrenuti kada je miš iznad bilo koje kontrole.

Koristite svojstvo `Event.LastMouseWheelDirection` da biste znali da li se točak pomera gore ili dole.

Za više informacija, pročitajte o rukovanju događajima kontrola.

Primer: Dodajte tekstualno polje na formu, a zatim dodajte ovaj pod-menadžer u prozor koda da biste omogućili korisniku da uvećava ili smanjuje prikaz kada pritisne taster `Ctrl` dok pomera točak miša gore i dole. Imajte na umu da će ovo raditi čak i kada je miš iznad tekstualnog polja:

```
Sub Form1_OnPreviewMouseWheel()  
If Keyboard.CtrlPressed Then  
d = Event.LastMouseWheelDirection  
Me.Width = Me.Width + 6 * d  
Me.Height = Me.Height + 6 * d  
TextBox1.Top = TextBox1.Top + 3 * d  
TextBox1.Left = TextBox1.Left + 3 * d  
TextBox1.Width = TextBox1.Width + 3 * d  
TextBox1.Height = TextBox1.Height + 3 * d  
EndIf  
EndSub
```

Form.OnShown

Ovaj događaj se pokreće nakon što se forma prikaže i sve kontrole se renderuju i spremne su za korišćenje svojih svojstava. Dakle, ako naidete na probleme sa izvršavanjem koda u globalnoj oblasti forme jer se neke kontrole još uvek ne renderuju, možete umesto toga koristiti ovaj događaj. Ali nemojte koristiti globalnu oblast i događaj `OnShown` zajedno, jer je redosled njihovog izvršavanja nepoznat, pa kada koristite događaj `OnShown`, stavite sav kôd za inicijalizaciju u njega i ostavite globalnu oblast praznom.

Za više informacija, pročitajte o rukovanju događajima kontrola.

Form.RemoveControl(controlName)

Uklanja datu kontrolu iz forme.

Primeri: Dodajte tekstualno polje, dugme, labelu i listu na formu, a zatim dodajte ovaj kôd u obrađivač događaja klika na dugme, koji će ukloniti tekstualno polje, labelu i listu iz formulara:

```
Me.RemoveControl(TextBox1)  
Me.RemoveControl(Label1)  
Me.RemoveControl(ListBox1)
```

Takođe možete koristiti petlju da biste uklonili sve kontrole (uključujući dugme):

```
ForEach _control In Me.Controls  
Me.RemoveControl(_control)  
Next
```

Form.RunGlobalTests() As Double

Pokreće funkcije testiranja napisane u datoteci `Global.sb` trenutnog projekta i prikazuje rezultate testiranja u tekstualnom polju `TxtTest` na trenutnom formularu. Ako formular ne sadrži tekstualno polje sa ovim imenom, ono će biti dodato tokom izvršavanja da bi se prikazali rezultati, sa

veličinom koja pokriva ceo formular.

Imajte na umu da će pozivanje metode `UnitTest.RunGlobalTests` pozvati metodu `Form.RunGlobalTests` forme prosleđenog kao argument.

Takođe imajte na umu da će pozivanje metode `UnitTest.RunAllTests` pokrenuti sve testove u projektu, uključujući i globalne testove, tako da ne morate ponovo da pozivate ovu metodu.

Za više informacija o tome kako pisati i debugovati funkcije za testiranje, pogledajte napomene o metodi `Form.RunTests` u sledećem odeljku.

Vraća: Broj pokrenutih testova. Ako ne postoji globalna datoteka ili ako ne sadrži nikakve funkcije za testiranje, ova metoda će vratiti 0.

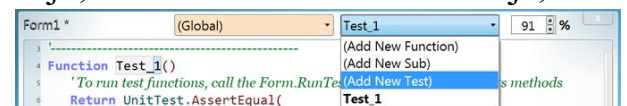
Form.RunTests() As Double

Pokreće funkcije za testiranje napisane u trenutnom obrascu i prikazuje rezultate testa u tekstualnom polju `TxtTest`. Ako obrazac ne sadrži tekstualno polje sa ovim imenom, biće dodato tokom izvršavanja da bi se prikazali rezultati, sa veličinom koja pokriva ceo obrazac.

Vraća: Broj pokrenutih testova ili 0 ako forma nema nikakve funkcije za testiranje.

Napomene:

- Da biste lakše pisali test funkcije, gornja desna padajuća lista uređivača koda nudi vam komandu „Add New Test“ – *Dodaj novi test*, i kada kliknete na nju, dodaće se šablon test funkcije, tako da možete lako izmeniti njeno ime i kod:



- Test funkcija mora da poštuje ova pravila:

1. Mora biti funkcija, a ne podfunkcija.

2. Njeno ime mora da počinje sa ``Test_``, kao što je ``Test_FindNames``.

3. Ne sme imati parametre.

4. Njena povratna vrednost treba da bude string koji sadrži rezultat testa, kao što je „Passed“ – *prošlo* ili „Failed“ – *neuspešno*. Ako vratite prazan string, ovaj rezultat će biti ignorisan.

Za praktične primere, pogledajte test funkcije napisane u projektu „UnitTest Sample“ u folderu sa primerima.

- Lista automatskog dovršavanja uređivača koda neće prikazivati imena test funkcija, ali ih i dalje možete ručno napisati i pozvati u kodu ako vam je to potrebno.

- Kada pišete test funkcije u datoteci `global.sb`, one će biti kompajlirane kao prijateljske/interne metode, što znači da ih ne možete pozivati ni iz jednog drugog projekta kada koristite svoj projekat kao biblioteku koda. Zato budite oprezni kada pišete biblioteku koda da ne počinjete ime bilo koje funkcije koja nije za testiranje sa prefiksom „Test_“.

- Koristite metodu `Form.RunGlobalTests` za pokretanje testova napisanih u datoteci `Global.sb`.

- Pozivanje metode `UnitTest.RunFormTests` sa formom prosleđenom kao argumentom pozivaće metodu `Form.RunTests`.

- Pozivanje metode `UnitTest.RunAllTests` pozivaće metodu `Form.RunTests` za sve forme projekta i prikazuje rezime rezultata u tekstualnom polju koje je prosleđeno kao argument, pored pozivanja metode `Form.RunGlobalTests` za roditeljski oblik tog tekstualnog polja.

- sVB debager može da ulazi u funkcije testiranja jednu za drugom, bez obzira da li su pozvane od strane `Form.RunTests`, `Form.RunGlobalTests`, `UnitTest.RunFormTests` ili metode `UnitTest.RunAllTests`. Takođe možete dodati tačke prekida unutar bilo koje test funkcije.

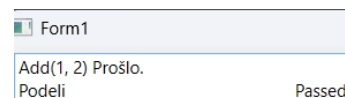
Primeri: Dodajte sledeće dve funkcije u formu:

```
Function Add(a, b)
Return a + b
```

```
EndFunction
Function Divide(a, b)
If b = 0 Then
Return ""
EndIf
Return a / b
EndFunction
```

Sada dodajte sledeće dve test funkcije da biste ih testirali:

```
Function Test_Add()
If Add(1, 2) = 12 Then
Return "Add(1, 2) Neuspešno!"
ElseIf Add(1, 2) = 3 Then
Return "Add(1, 2) Prošlo."
EndIf
EndFunction
Function Test_Divide()
Return UnitTest.AssertTrue(
{
Divide("4", "2") = 2,
Divide(4, 2) = 2,
Math.Round2(Divide(5, 3), 2) = 1.67,
Divide(2, 0) = "",
Divide(0, 0) = ""
},
"Podeli"
)
EndFunction
```



Imajte na umu da funkcija Test_Add vraća rezultat testa ručno, dok funkcija Test_Divide koristi jednu od UnitTest metoda za proveru vrednosti testa i vraćanje formatiranog stringa koji sadrži rezultate, što olakšava pisanje testova.

Sada, da biste pokrenuli ova dva testa, dodajte ovu izjavu na početak koda forme:

```
Me.RunTests()
```

Pritisnite F5 da biste pokrenuli projekat. Forma će prikazati ove rezultate:

Form.Show()

Prikazuje formu na ekranu, ako je učitana, ali još nije prikazana, ili ako je skrivena.

Primer: Možete videti ovu metodu u akciji u projektu „Random Buttons“ – *Slučajna dugmad* u folderu sa primerima. Koristi se u obradi događaja Button2_OnClick da bi se prikazao nova forma koju kreiramo tokom izvršavanja:

```
Sub Button2_OnClick()
form2 = Forms.AddForm("form2")
form2.Text = "Form2"
newButton = CreateRndButton(form2)
newButton.OnClick = Button2_OnClick
form2.Show()
EndSub
```

Form.ShowChildForm(childFormName, argsArr) As Form

Prikazuje formu koja ima dato ime kao podređeni obrazac trenutnog obrasca. Ova metoda će uraditi sledeće:

- Učitati dizajn datog obrasca iz njegove xaml datoteke.
- Proslediti date podatke argsArr svojstvu Form.ArgsArr ciljnog obrasca.
- Izvršiti kôd napisan u globalnoj oblasti datoteke koda ciljnog obrasca. Možete koristiti Form.ArgsArr u ovoj globalnoj oblasti za inicijalizaciju obrasca. Imajte na umu da se globalni kôd

izvršava samo kada se obrazac otvori prvi put ili nakon što se zatvori, a zatim ponovo otvori. Neće se izvršiti kada sakrijete ili minimizirate obrazac, a zatim ga ponovo prikažete.

- Prikažite ciljni obrazac kao podređeni dijaloški obrazac. Podređeni obrazac je obrazac koji se uvek pojavljuje preko svog nadređenog obrasca, čak i kada je nadređeni obrazac aktivni obrazac. To znači da podređeni obrazac ne blokira nadređeni obrazac kao što to čini modalni dijaloški obrazac. Ovo omogućava korisniku da radi sa dva obrasca istovremeno. Prozor za pronalaženje i zamenu je dobar primer podređenog obrasca, gde možete interagovati sa tekstom koji pretražujete dok je prozor za pronalaženje i zamenu i dalje prikazan. Korisnik može minimizirati ili zatvoriti samo podređeni obrazac, ali minimiziranje nadređenog obrasca će minimizirati i podređeni obrazac, a zatvaranje nadređenog obrasca će takođe zatvoriti podređeni obrazac.

- Pokrenite događaj OnShown obrasca ako ima obrađivač.

Parametri:

- *childFormName*: Ime podređenog obrasca.

- *argsArr*: Bilo koji dodatni podatak, niz ili dinamički objekat koji želite da prosledite formi. Biće sačuvan u svojstvu Form.ArgsArr podređene forme, tako da ga možete koristiti kako želite.

Vraća: Ime podređene forme.

Primer: Možete videti ovu metodu u akciji u projektu „sVB Notepad“ u folderu sa primerima. Koristi se u obrađivaču događaja MenuFind_OnClick da bi se prikazao obrazac za pretragu, pod nazivom „FrmFind“, kao podređeni obrazac glavne forme. Prosleđujemo polje za tekst, pod nazivom „TxtEditor“, u ArgsArr, tako da frmFind može da izvrši operacije pretrage u sadržaju tog polja za tekst.

```
Sub MenuFind_OnClick()  
Me.ShowChildForm("FrmFind", TxtEditor)  
EndSub
```

Form.ShowDialog() As DialogResult

Prikazuje trenutni obrazac na ekranu kao modalni dijalog, tako da korisnik mora prvo da ga zatvori da bi mogao da pristupi drugim oblicima vaše aplikacije.

Vraća: Rezultat dijaloga, koji može biti string koji predstavlja tip dugmeta na koje je korisnik kliknuo, kao što su U redu, Da, Ne, ... itd. Lakše je koristiti članove DialogResult umesto string imena dugmadi kako bi se izbegle greške u kucanju.

Takođe je moguće vratiti niz ili dinamički objekat da biste dobili više informacija iz dijaloga. Ove informacije vraćate postavljanjem svojstva Form.DialogResult u dijaloškom obrascu.

Primer: Dodajte dugme na formu i dodajte kod u formu:

```
N = 1  
Sub Button1_OnClick()  
N = N + 1  
f = Forms.AddForm("form" + N)  
f.Text = "Form " + N  
oKButton = f.AddButton(  
"OKButton", 130, 300, 100, 40)  
oKButton.Text = "OK"  
oKButton.OnClick = DialogButton_OnClick  
cancelButton = f.AddButton(  
"CancelButton", 250, 300, 100, 40)  
cancelButton.Text = "Otkazi"  
cancelButton.OnClick = DialogButton_OnClick  
Me.ShowMessage(  
"Korisnik je pritisnuo " + f.ShowDialog()  
+ " da zatvori " + f,
```

```

"Rezultat"
)
EndSub
Sub DialogButton_OnClick()
    _button = Event.SenderControl
    _form = _button.ParentForm
    If _button.Name = "OKButton" Then
        _form.DialogResult = DialogResult.OK
    EndIf
    _form.Close()
EndSub

```

Pritisnite F5 da biste pokrenuli projekat. Svaki put kada kliknete na dugme, biće prikazan novi dijalog sa dugmadima OK – *U redu* i Cancel – *Otkazi*.

Kada zatvorite obrazac klikom na bilo koje od njih ili korišćenjem dugmeta x na obrascu, pojaviće se poruka koja vam govori na koje dugme ste kliknuli da biste zatvorili obrazac.

Form.ShowMessage(message, title)

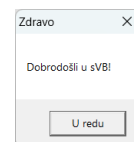
Prikazuje dijalog okvira za poruku na trenutnoj formi, koji postaje njegov vlasnik. Okvir za poruku će prikazati navedenu poruku i dugme U redu za zatvaranje prozora.

Imajte na umu da možete prikazati prozor sa porukom Da/Ne/Otkazi i interagovati sa korisničkim izborom koristeći metodu Dialogs.MsgBox.

Parametri:

- *message*: Tekst koji će se prikazati u prozoru sa porukom.
- *title*: Naslov koji će se prikazati u dijalog prozoru. *Primer:*

```
Me.ShowMessage("Dobrodošli u sVB!", "Zdravo")
```



Form.Text As String

Dobija ili postavlja naslov forme.

Form.Topmost As Boolean

Dobija ili postavlja da li je forma najviši prozor koji se uvek pojavljuje preko svih ostalih prozora na radnoj površini, čak i kada nije aktivni prozor.

Form.Validate() As Boolean

Poziva metodu Control.Validate za svaku kontrolu na trenutnoj formi. Proces se zaustavlja na prvoj kontroli sa greškama i vraća vrednost false.

Imajte na umu da morate pozvati ovu metodu kada korisnik klikne na dugme koje obrađuje ulazne podatke, jer može ostaviti neke kontrole praznim i nikada ne postaviti fokus na njih, pa nikada nisu pokrenuli događaj OnLostFocus, stoga još nisu validirane, dok vaša logika zahteva da neke od njih imaju podatke i ne mogu biti prazne. Provera svojstva Error ovih kontrola neće rešiti ovaj problem, jer još nisu proverene na greške, pa će izgledati u redu. Pozivanje Form.validate će rešiti ovaj problem, jer primorava svaku kontrolu da pokrene svoj događaj OnLostFocus, tako da će kontrole koje imaju obrađivače za ovaj događaj biti primorane da validiraju svoj unos.

Vraća: True ako su sve kontrole validne, ili False u suprotnom.

Primer: Ova metoda se koristi u dugmetu validate2 u projektu Validation u folderu samples, kako bi se osiguralo da su sve kontrole validne:

```
If Me.Validate() Then  
Me.ShowMessage("OK", "Validation")  
EndIf
```